

INFORME DE LABORATORIO — TALLER 2

Desarrollo en Flutter con Asincronía (Future, async/await), Manejo de Tiempo con Timer y Concurrencia con Isolate

Estudiante:	Michael Stiven Vasco Cárdenas
Código Estudiantil:	230231047
Asignatura:	Electiva Profesional I
Docente:	Ingeniería de Sistemas
Repositorio GitHub:	https://github.com/BreiK-at/Taller-flutter
Rama del Taller:	feature/taller2
Fecha de Entrega:	Octubre 2026

Tuluá, Valle del Cauca, Colombia
2026

1. Objetivo del Taller

Construir una aplicación reactiva en Flutter que demuestre el uso adecuado de mecanismos de ejecución no bloqueante: asincronía con **Future** y `async/await`, gestión de tareas recurrentes basadas en tiempo mediante **Timer**, y procesamiento concurrente intensivo de CPU con **Isolate.spawn**, garantizando en todo momento la fluidez de la interfaz (60 FPS). Asimismo, se aplica la metodología **Git Flow** con ramas protegidas y Pull Requests.

2. Comparativa Técnica de Mecanismos de Concurrency

Mecanismo	Caso de Uso Principal	¿Bloquea UI?	Mecanismo Interno
Future (<code>async/await</code>)	Operaciones I/O bound (peticiones HTTP, base de datos, lectura de archivos, retardos simulados).	No	Agenda callbacks en el <i>Event Queue</i> del hilo principal. Permite que el framework continúe renderizando frames.
Timer (<code>dart:async</code>)	Control de intervalos temporales (cronómetros, cuentas regresivas, debouncing de búsquedas).	No	Encola eventos repetitivos cada tick en el Event Loop. Exige cancelación obligatoria en <code>dispose()</code> para evitar fugas.
Isolate (<code>dart:isolate</code>)	Tareas CPU-bound pesadas (cálculo matemático exhaustivo, cifrado masivo, procesamiento de video/imágenes).	No	Crea un hilo nativo del sistema operativo con heap de memoria propio. Comunica resultados exclusivamente vía <code>SendPort</code> / <code>ReceivePort</code> .

3. Fragmentos de Código Clave Implementados

3.1 Asincronía con `Future.delayed` y Manejo de Estados:

```
Future<List<Map<String, dynamic>>> consultarTareasRemotas({bool forzarError = false}) async {
  print('[1. ANTES DE LLAMAR AL SERVICIO] Solicitando datos remotos...');
  await Future.delayed(const Duration(milliseconds: 2500)); // Retardo no bloqueante
  if (forzarError) throw Exception('Fallo de conexion al servidor remoto');
  print('[3. DESPUES DE RECIBIR LA RESPUESTA] Datos recibidos exitosamente.');
```

```
  return _tareasSimuladas;
}
```

3.2 `Isolate.spawn` con Comunicación por Puertos y Medición con `Stopwatch`:

```
static void _tareaPesadaTopLevel(SendPort sendPort) {
  final stopwatch = Stopwatch()..start();
  int totalPrimos = 0;
  for (int i = 2; i <= 30000000; i++) {
    if (_esPrimo(i)) totalPrimos++;
  }
  stopwatch.stop();
  sendPort.send({'tiempoMs': stopwatch.elapsedMilliseconds, 'primos': totalPrimos});
}
```

4. Evidencias de Asincronía con Future y async/await

Se diseñó la pantalla AsyncScreen implementando una máquina de estados visual reactiva (Inicial, Cargando, Éxito, Error) acompañada de una consola en vivo integrada para verificar el orden de ejecución:

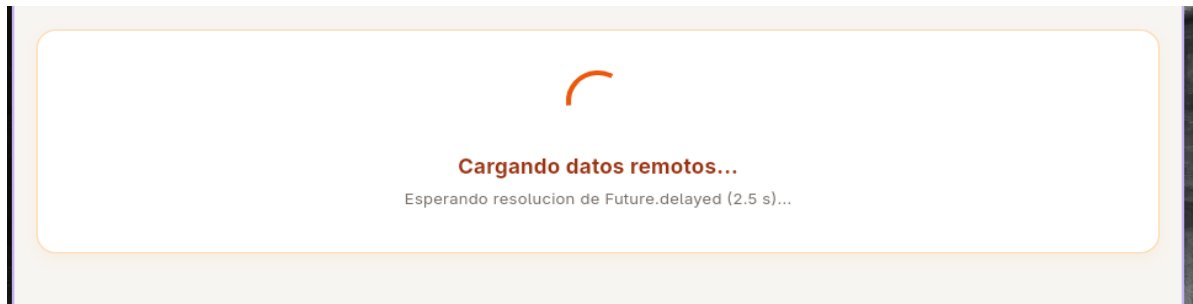


Figura 1: Estado Cargando — Feedback visual inmediato con *CircularProgressIndicator* y botones deshabilitados para evitar múltiples llamadas concurrentes.

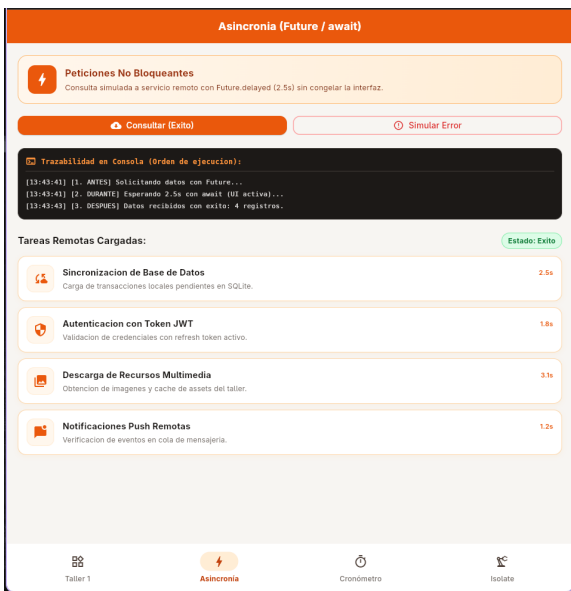


Figura 2: Estado Éxito

4 registros obtenidos remotamente y renderizados en tarjetas individuales con badge de prioridad y registro en consola.

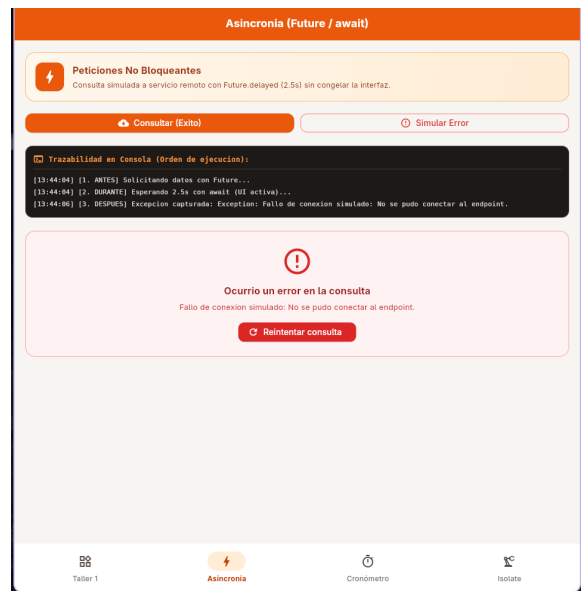


Figura 3: Estado Error Controlado

Captura en bloque try/catch con tarjeta de alerta roja, mensaje descriptivo y botón de reintento.

Trazabilidad del Orden de Ejecución en Consola:

- [1. ANTES]: El usuario activa el botón; se dispara el método asíncrono y se cambia de inmediato el estado a *Cargando*.
- [2. DURANTE]: Durante los 2.5 segundos de *Future.delayed*, el hilo principal continúa procesando animaciones sin retraso.
- [3. DESPUES]: Tras resolverse el *Future*, se muta el estado con *setState()* desplegando la colección de datos o capturando el error.

5. Evidencias del Cronómetro Digital con Timer

Se implementó TimerScreen utilizando Timer.periodic con intervalo de 100 milisegundos para garantizar precisión y respuesta visual fluida en tiempo real:

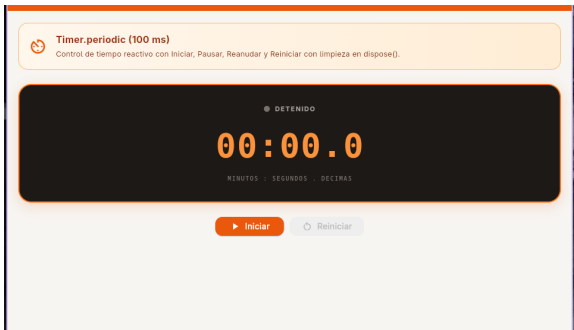


Figura 4: Estado Inicial (Detenido)
Display en 00:00.0 y botón activo Iniciar.

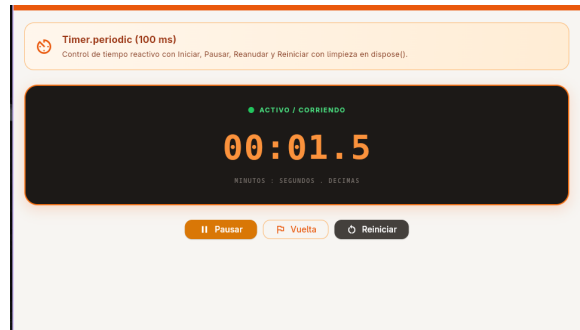


Figura 5: Estado Activo (Corriendo)
Tiempo transcurriendo a 100 ms por tick, botones Pausar y Vuelta.

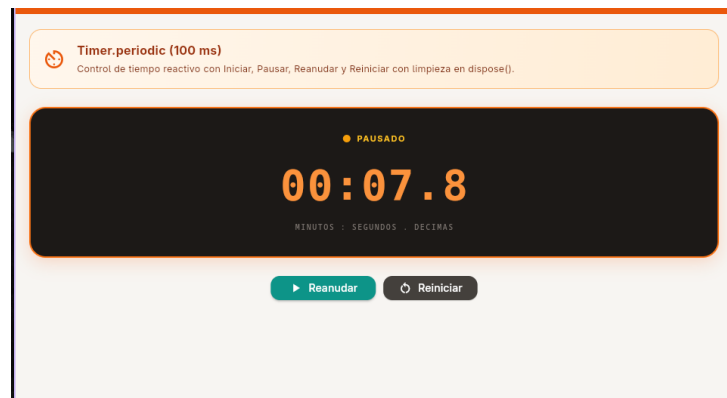


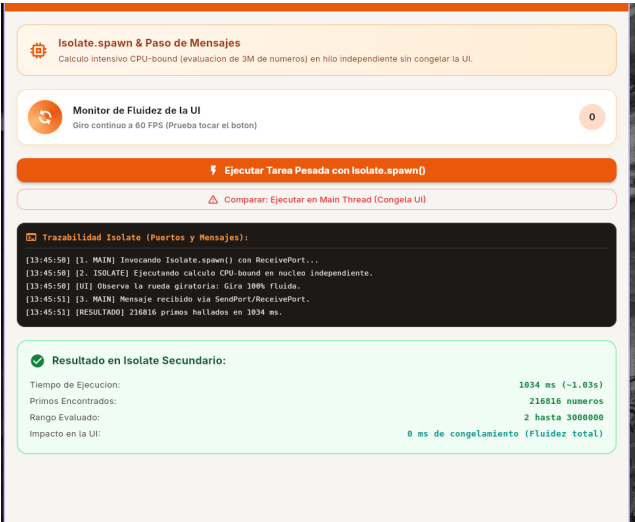
Figura 6: Estado Pausado con Historial de Vueltas — Cronómetro detenido en 00:07.8 mediante `_timer?.cancel()` conservando el acumulador y habilitando Reanudar y Reiniciar.

Gestión Estricta del Ciclo de Vida y Limpieza de Recursos:

- **Prevención de Fugas de Memoria (Memory Leaks):** Si un widget es desmontado del árbol mientras un Timer.periodic sigue corriendo, el callback intentará invocar `setState()` sobre un elemento inactivo arrojando excepciones en Flutter. En la solución se sobrescribió `dispose()` ejecutando obligatoriamente `_timer?.cancel()`.
- **Precisión Temporal y Formateo:** Se maneja un contador en milisegundos netos, calculando minutos, segundos y décimas con operaciones enteras para evitar acumulación de errores de punto flotante.

6. Evidencias de Concurrencia Pesada con Isolate

Se construyó IsolateScreen para someter a prueba la UI ante una tarea CPU-bound masiva: comprobar la primalidad de 3.000.000 de enteros:



Demostración de No Congelamiento:

- **Monitor de Fluidez:** Se integró un icono giratorio continuo accionado por `AnimationController.repeat()`.
- **UI Interactiva:** El usuario puede pulsar el contador de clicks en tiempo real mientras el Isolate calcula.
- **Resultado del Worker:** 216.816 primos hallados en 1.034 ms.
- **Paso de Mensajes:** El resultado viajó del hilo secundario al principal encapsulado en un Map vía `SendPort`.
- **Comparación Directa:** La misma operación en el Main Thread congela el renderizado causando *jank* severo de más de 1 segundo.

Figura 7: Cómputo completado en Isolate sin interrupción de los 60 FPS de la interfaz gráfica.

7. Trazabilidad del Flujo de Trabajo (Git Flow)

Pull Request	Ramas	Estado	Descripción
PR #1	feature/taller1 -> dev	Merged	Integración inicial de pantallas y widgets de Taller 1.
PR #2	dev -> main	Merged	Pase a producción de release inicial Taller 1.
PR #3	feature/taller2 -> dev	Merged	Integración de módulos Future, Timer e Isolate.
PR #4	dev -> main	Merged	Pase a producción final de Taller 2 con documentación.

8. Conclusiones

- La asincronía con `Future` y `async/await` es la vía óptima para tareas I/O dependientes de la red o disco, delegando la espera al Event Loop sin sobrecargar los núcleos de la CPU.
- Los `Timer` deben administrarse con extrema rigurosidad dentro del ciclo de vida del widget, implementando siempre `cancel()` en `dispose()`.
- Para tareas intensivas de CPU, el uso de `Isolate.spawn` es indispensable para evitar el bloqueo del hilo de interfaz gráfica, aislando la memoria y garantizando 60 FPS estables.